

При обучении на одном девайсе мы ограничены аппаратными возможностями одного GPU.

Во-первых, мы ограничены по памяти. При обучении нам необходимо хранить параметры модели, градиенты, состояния оптимизатора и активации. Это очень сильно ограничивает размер моделей.

Во-вторых, мы ограничены по вычислительной мощности. Допустим, у нас нет проблемы памяти и мы собрались обучать LLM. Для предобучения модели нам потребуется $4.8 \cdot 10^{26}$ FLOPs. При обучении на одной H100 на пике производительности в точности `float16` это займёт ~ 7600 лет. К сожалению, мы слишком непоседливы, чтобы ждать столько времени.

Чтобы решить эти проблемы, и обучать по настоящему большие модели за адекватное время, необходимо использовать тысячи видеокарт.

Однако, несколько GPU сами по себе не собираются в одну большую видеокарту. У каждого девайса — своя память и свои вычислительные блоки. Если тензор находится в памяти одного девайса, то другой девайс не может читать его прямым образом как из своей памяти. Нам необходимо передавать данные между GPU. Передача данных между GPU называется **коммуникациями**.

1 Связь нескольких GPU

Некоторые коммуникации могут быть быстрыми, а некоторые — медленными. При этом, их необходимо выстроить таким образом, чтобы нагрузка на GPU была распределена равномерно и не было простоя в вычислениях из-за ожидания необходимых данных.

Чтобы понять, почему коммуникации могут отличаться по скорости, необходимо сначала разобраться, как GPU физически связаны между собой.

1.1 Устройство GPU-ноды

На первом уровне иерархии кластера GPU находится нода — небольшой сервер, внутри которого установлено несколько GPU, обычно 8. Такая нода содержит CPU (обычно даже несколько CPU-сокетов), оперативную память, GPU и сетевые карты.

CPU управляет запуском вычислений, подготавливает данные, вызывает кернелы и коммуникационные операции. Оперативная память CPU — это хост-память. Она также может использоваться, например, для выгрузки тех данных, которые не способны в текущий момент времени поместиться в GPU.

Сетевая карта, NIC, соединяет ноду с остальным кластером. Для распределенного обучения важно, где находится NIC относительно GPU.

Важна также NUMA-локальность. Если в ноду несколько CPU-сокетов, то память и девайсы могут быть ближе к одному сокету и дальше от другого. Поэтому два девайса одной ноды не всегда эквивалентны с точки зрения

доступа к CPU памяти и NIC.

1.2 Внутринодовая связь GPU

Внутринодовая связь — это связь между GPU внутри одного сервера. Обычно она быстрее и дешевле, чем связь между разными нодами.

Самый базовый способ соединения GPU внутри ноды — PCIe шина. Через него GPU может обмениваться данными с CPU, памятью хоста, сетевой картой и другим GPU. Это универсальное соединение, но для интенсивного обмена между GPU его пропускной способности может быть недостаточно.

Для более быстрого обмена между GPU используется соединение NVLink. Он дает более высокую пропускную способность и лучше подходит для частых коммуникаций между девайсами. NVLink подключается к GPU отдельно от PCIe и соединяет GPU только попарно.

Возникает проблема топологии: одна GPU может быть соединена с несколькими соседями, но не обязательно с несколькими GPU в ноде. Тогда передача между некоторыми парами GPU может идти через промежуточные устройства.

На помощь приходит NVSwitch, его можно рассматривать как коммутатор для NVLink. Мы можем подключить GPU к NVSwitch через NVLink и обмениваться между всеми девайсами внутри ноды.

Кроме физического соединения важно использовать P2P (peer-to-peer) связь. Без использования P2P, данные будут проходить через CPU RAM, что очень дорого:

$$\text{GPU}_0 \rightarrow \text{CPU} \rightarrow \text{GPU}_1,$$

а при использовании P2P:

$$\text{GPU}_0 \rightarrow \text{GPU}_1$$

1.3 Межнодовая связь

Обычно, ноды объединяют по 8 GPU. Этого всё еще мало, нам нужен кластер из тысяч GPU. Чтобы этого достичь, необходимо для начала наладить межнодовую связь.

Упрощенно межнодовую связь можно представить так:

$$\text{GPU}_0 \rightarrow \text{CPU}_0 \text{ memory} \rightarrow \text{NIC}_0 \rightarrow \text{Сеть} \rightarrow \text{NIC}_1 \rightarrow \text{CPU}_1 \text{ memory} \rightarrow \text{GPU}_1$$

Как мы уже обсуждали, передавать данные через CPU память очень дорого. Поэтому, чтобы избежать этого, будем передавать данные напрямую от девайса к NIC, используя **удаленный прямой доступ к памяти (remote direct memory access, RDMA)**. Тогда схема будет выглядеть короче:

$$\text{GPU}_0 \rightarrow \text{NIC}_0 \rightarrow \text{Сеть} \rightarrow \text{NIC}_1 \rightarrow \text{GPU}_1$$

Сетевые интерфейсы NIC — это аппаратные компоненты ноды, которые являются выходом в сеть. При перемещении данных по сети при обучении LLM обычно используют Infiniband.

1.4 Топология кластера

Объединение нод в одну сеть, **кластер**, можно рассматривать как граф. Вершины — это ноды, а ребра — сетевые соединения между ними.

Если рассматривать более подробно, то можно заметить, что каждая нода — тоже граф, только на этот раз вершины — GPU.

Расстояние между двумя GPU внутри ноды значительно меньше, чем расстояние между двумя GPU в разных нодах. Межнодовая связь значительно сложнее, а значит дороже, чем внутринодовая.

Но даже между нодами расстояние может быть разное. Некоторые ноды могут физически находиться на одном стеллаже и быть подключены к одному сетевому коммутатору. Другие ноды могут обмениваться данными через несколько слоёв сетевой инфраструктуры.

В больших кластерах важна топология сети. Например:

- fat-tree,
- dragonfly,
- torus

Мы не будем их разбирать, однако важно понимать, что топология определяет, какие коммуникации будут дешевыми, а какие — дорогими, потому что это напрямую влияет на распределенное обучение.

Если стратегия параллелизма требует частого обмена большими тензорами между удаленными нодами, то обучение может упереться в стоимость коммуникаций.

1.5 Стоимость коммуникаций

Основные метрики, которые оценивают стоимость коммуникаций — это задержка передачи α и пропускная способность B .

Простейшая модель времени коммуникации имеет вид:

$$T_{\text{comm}}(n) \approx \alpha + \frac{n}{B},$$

где n — размер передаваемых данных. Из этой формулы видно, что при малых сообщениях в стоимости коммуникации доминирует задержка, а при больших — скорость пропускной способности. Запускать много мелких коммуникаций — плохо, лучше объединять их в большие пакеты.



Рис. 1: Схема fat-tree топологии.

2 Коммуникации

В распределенном обучении говорят не просто про девайсы, а про ранги. Ранг — это отдельный участник распределенного обучения. Часто (но не всегда) один ранг соответствует одному GPU.

Существуют глобальные ранги и локальные ранги. Глобальный ранг — это ранг во всем кластере. Локальный ранг — это ранг внутри ноды.

Каждый ранг хранит свои локальные тензоры и выполняет свою часть вычислений, в зависимости от стратегии параллелизма. Паттерны, которые устанавливают, какие данные должны быть переданы между рангами, называются коллективными коммуникациями.

2.1 Point-to-point коммуникации

Перед рассмотрением коллективных коммуникаций, разберем простейшую схему обмена данными — между двумя GPU.

Такие операции называются point-to-point операциями: один ранг отправляет буфер данных с помощью операции `send`, а второй принимает его с помощью операции `recv`.

Point-to-point операции просты по смыслу, но требуют аккуратной синхронизации. Если один ранг ждет данные, а другой их не отправил, то этот ранг будет ждать их вечно. Такая проблема называется deadlock.

Поэтому обычно point-to-point операции используют внутри коллективных коммуникационных операций, где порядок действий заранее фиксирован (хотя, такие операции также могут использоваться в некоторых типах параллелизма).

2.2 Коллективные коммуникации

В коллективных коммуникациях участвует уже не пара рангов, а целая группа (называемая процесс-группой). Эти операции — шаблоны. Но именно такие шаблоны часто и повторяются в различных стратегиях распределенного обучения.

Рассмотрим некоторые из этих операций.

AllReduce AllReduce — операция, которая выполняет редукцию данных между рангами в группе.



Рис. 2: Схема AllReduce.

Каждый ранг группы получает результат редукции.

Broadcast `Broadcast` — операция, которая копирует тензор из корневого ранга на остальные ранги.

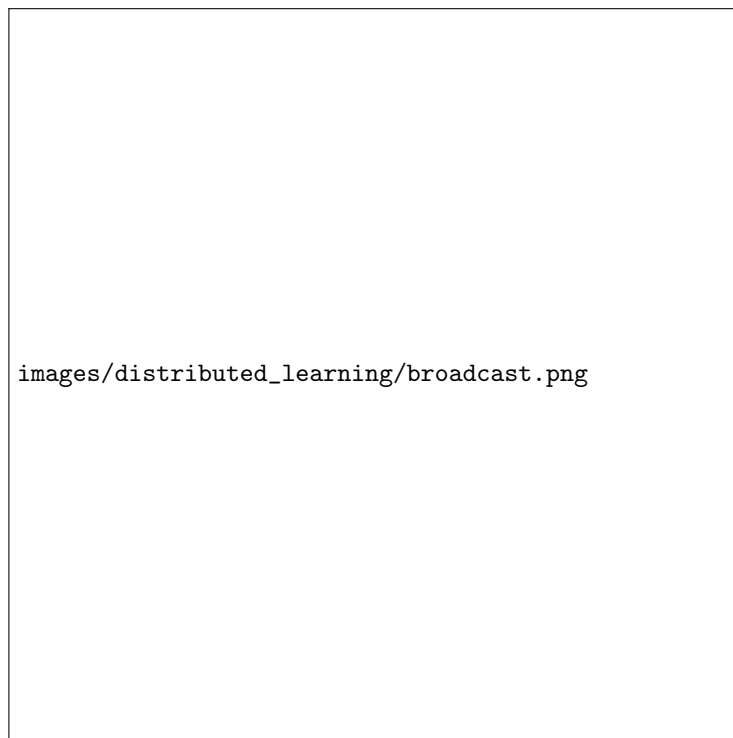


Рис. 3: Схема Broadcast.

AllGather `AllGather` собирает данные с каждого ранга и раздает полный результат каждому рангу.

В итоге каждый ранг получает тензор, собранных из отдельных частей со всех рангов.

Reduce Операция `Reduce` выполняет ту же операцию, что и `AllReduce`, но итоговый результат помещается только на один ранг.

ReduceScatter `ReduceScatter` объединяет редукцию и разбиение результата.

Сначала локальные тензоры редуцируются, например, суммируются

$$z = \sum_{i=0}^{N-1} x_i$$

после чего z разбивается на несколько частей

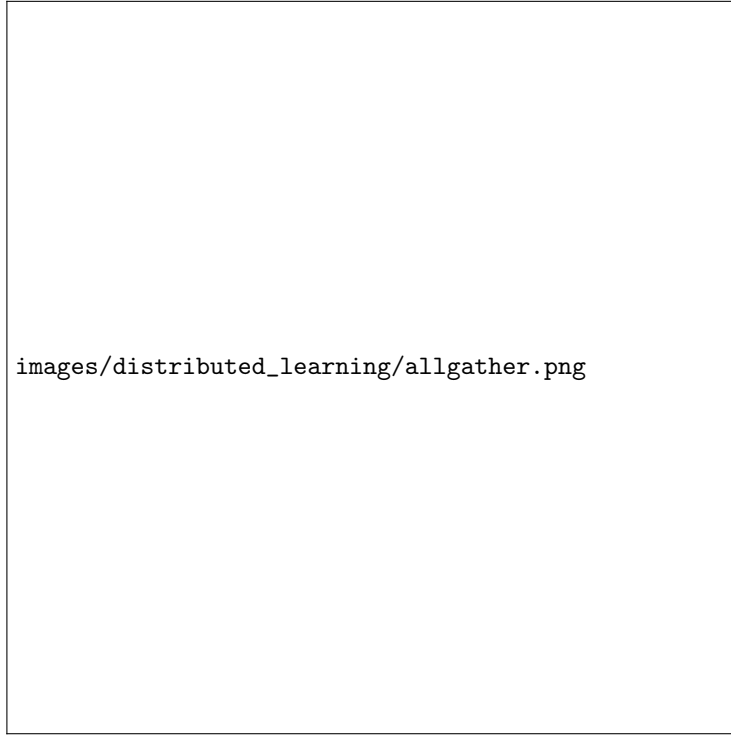


Рис. 4: Схема AllGather.

$$z = [z_0, z_1, \dots, z_{N-1}]$$

и каждый ранг i получает только свою часть $y_i = z_i$.

AlltoAll **AlltoAll** — это операция, при которой каждый ранг отправляет разные части своего тензора всем остальным рангам в группе.

Пусть на ранге i хранится тензор, разбитый на N частей:

$$x_i = [x_{i,0}, x_{i,1}, \dots, x_{i,N-1}]$$

Часть $x_{i,j}$ предназначена для ранга j . После **AlltoAll** ранг j получает тензор, состоящий из следующих частей:

$$y_j = [x_{0,j}, x_{1,j}, \dots, x_{N-1,j}]$$

AlltoAll — одна из самых тяжелых коллективных операций, потому что количество обменов быстро растет с числом рангов. Схему этой коллективной операции можно представить в виде полного графа, где каждая вершина связана с каждой другой.

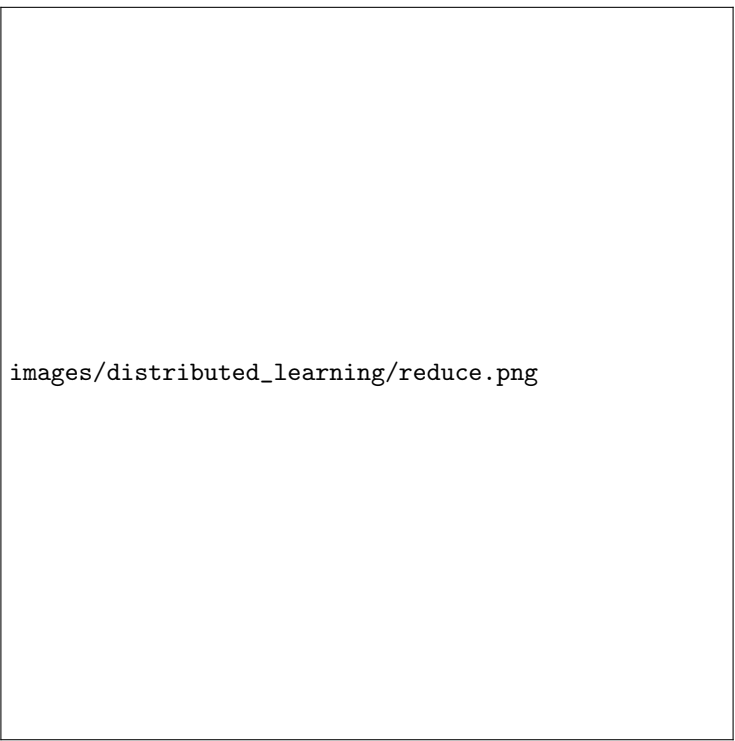


Рис. 5: Схема Reduce.

2.3 Объединение коллективных коммуникаций

Некоторые коллективные коммуникации, выполняемые подряд, можно объединить в одну. Например, последовательное выполнение операций **Reduce** и **Broadcast** можно заменить на **AllReduce**.

Причем такая замена будет выгодна, поскольку **AllReduce** выполняется быстрее, чем последовательное выполнение сначала **Reduce** и следом **Broadcast**.

Другое полезное объединение коммуникаций:

$$\text{AllReduce} = \text{ReduceScatter} + \text{AllGather}$$

ReduceScatter считает редуцированный результат, но оставляет на каждом ранге только его часть. Затем **AllGather** собирает эти части обратно и раздает полный результат всем рангам.

3 NCCL

Всё, что мы рассмотрели в предыдущем разделе, является лишь теоретическими моделями операций коммуникации. Для их эффективного выпол-



Рис. 6: Схема ReduceScatter.

нения необходим инструмент, который будет способен настроить общение между GPU по заданному шаблону, учитывая топологию кластера, скорость внутринодовой и межнодовой связи.

Для этого используется библиотека коммуникаций **NVidia Collective Communication Library (NCCL)**. Она используется как бэкенд в системах распределенного обучения, реализуя рассмотренные ранее коллективные коммуникации (но не ограничиваясь только на них).

Одна и та же операция, например **AllReduce**, может выполняться разными способами даже в одном кластере, и в зависимости от размера пакета, NCCL будет минимизировать модель стоимости коммуникаций.

Так, для выполнения такой операции, NCCL берет на себя задачу выбора протокола передачи данных, определяет путь передачи от девайса к девайсу, разбивает сообщение на части, запускает кернелы коммуникации и синхронизирует передачу данных каждым шагом, чтобы не допустить ошибок или дедлоков.

NCCL — высокоуровневая библиотека коммуникаций, API которой мы можем использовать напрямую или, например, через PyTorch в виде модуля `torch.distributed`.

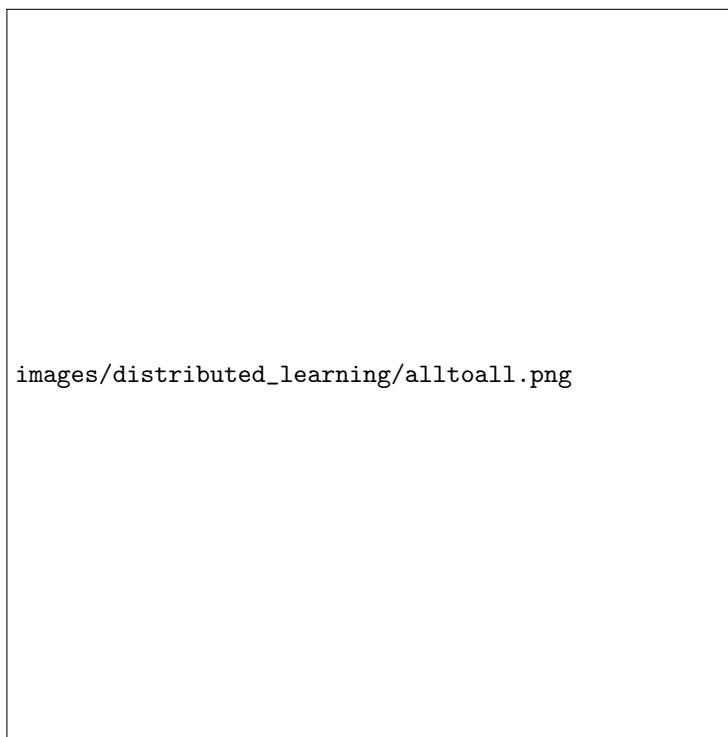


Рис. 7: Схема AlltoAll.

3.1 Модель исполнения

Основные сущности в NCCL — это ранги, стримы и коммуникатор. Что такое ранг мы уже знаем.

Коммуникатор — это объект, который помогает процесс-группе рангов выполнять коммуникационные операции друг с другом. Если **AllReduce** вызывается на коммуникаторе 8 рангов, то в операции участвуют только эти ранги.

Это может быть удобно, например, для того, чтобы явным образом отделить использование внутринодовых и межнодовых коммуникаций, что особенно важно при построении сетки стратегий параллелизма.

Стрим — это обычный CUDA стрим, который задает порядок исполнения операций на GPU. Операции коммуникации — это тоже кернелы, которые запускаются на GPU и требуют как ресурсы памяти, так и SM.

Мы запускаем коммуникации в отдельном стриме для того, чтобы выполнять их асинхронно и скрыть задержку коммуникаций под стримом вычислений. Если бы мы запускали все операции в одном стриме, то они бы выполнялись последовательно, что привело бы к простою или недоиспользованию GPU.

За счёт CUDA-событий мы можем синхронизировать работу стрима вычислений и стрима коммуникаций: очевидно, что нам не стоит начинать вычисление слоя до того, как мы полностью соберем веса этого слоя.

3.2 Коммуникационные каналы

NCCL не обязан передвигать большой тензор как один монолитный блок. Сообщения разбиваются на части, которые могут передаваться параллельно. Эти потоки передачи данных называются **коммуникационными каналами**.

Важно: коммуникационный канал — это логическая единица, а не физический кабель. Через один NVLink может проходить несколько коммуникационных каналов, также и один коммуникационный канал может передавать данные по нескольким физическим кабелям.



Рис. 8: Визуализация стратегии разделения данных NCCL между каналами.

Для больших сообщений большое число каналов может насытить пропускную способность. Но каждый канал требует вычислительных ресурсов GPU. Увеличивая число каналов, мы можем уменьшить время операций коллективных коммуникаций, но взамен замедлим выполнение кернелов на вычислительном стриме.

3.3 Протоколы передачи данных

Чтобы увеличить пропускную способность при передаче больших тензоров, мы можем передавать данные крупными блоками. Для обеспечения корректного порядка и видимости данных необходимо использовать барьеры памяти (memory fences).

Такой протокол передачи данных называется **Simple**. Он увеличивает утилизацию пропускной способности, но взамен также увеличивает задержку выполнения операции из-за барьеров-памяти.

Вспомним модель стоимости коммуникации:

$$T = \alpha + \frac{n}{B}$$

При большом значении n , то есть размера пакета, это выгодный протокол. При малом — нет.

Для работы с сообщениями малого размера используется протокол **LL** (**low latency**). Вместо использования барьеров памяти, LL полагается на синхронизации на основе передаваемых флагов. Они показывают, что данные валидны и получатель может продолжать работу. На каждые 4 байта данных передается 4 байта флага.

Simple — это одна крайность, полезная при пересылке больших пакетов, но увеличивающая задержку выполнения операции. LL — другая крайность, полезная при пересылке малых пакетов, но снижающая полезное использование пропускной способности по сути в два раза.

Компромиссный вариант — протокол **LL128**. Как и LL, этот протокол использует синхронизацию на основе флагов, однако на этот раз передает не 8, а 128 байт (отсюда и название). Из этих 128 байт, 8 байт занимает флаг, а 120 — полезная нагрузка.

Однако LL128 не всегда возможно использовать из-за высоких требований к оборудованию.

NCCL выбирает протоколы за нас, однако полезно понимать ограничения своей системы, чтобы оценивать какие операции будут занимать много времени и оптимизировать их до использования коммуникаций.

3.4 Алгоритмы коммуникаций

Одна и та же коллективная операция может быть выполнена разными алгоритмами. Порядок обмена данными может быть разным. Рассмотрим основные алгоритмы коммуникаций.

Ring При использовании алгоритма **Ring** ранги образуют кольцо. Например, для четырех девайсов такое кольцо будет выглядеть следующим образом:

$$\text{GPU}_0 \rightarrow \text{GPU}_1 \rightarrow \text{GPU}_2 \rightarrow \text{GPU}_3 \rightarrow \text{GPU}_0.$$

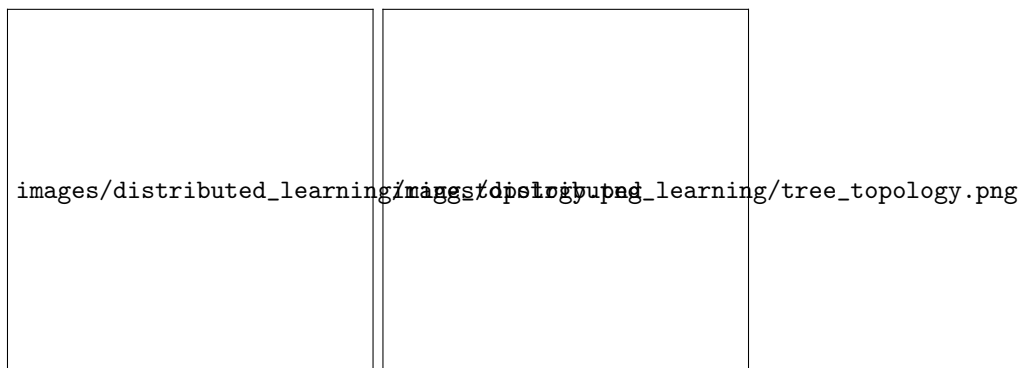


Рис. 9: Схемы алгоритма коммуникации по кольцу (слева) и коммуникации по дереву (справа).

Каждый ранг передает данные следующему рангу и получает данные от предыдущего. Большой тензор разбивается на части и эти части двигаются по кольцу, пока не совершат один полный обход.

Кольцевой алгоритм позволяет очень удачно утилизировать пропускную способность и при этом не раздувать память, которая выделяется под буферы.

Однако, количество шагов растет с увеличением количества рангов. Поэтому при большом количестве рангов возможна высокая задержка коммуникации.

Tree В древовидном алгоритме ранги организуются в дерево. Данные сначала поднимаются от листьев к корню, где выполняется редукция, а следом результат распространяется обратно вниз по дереву.

Такой алгоритм может иметь меньше последовательных шагов, чем кольцо, а глубина дерева растет как $O(\log N)$, поэтому в операциях, где участвует большое количество рангов, дерево может быть эффективнее, чем кольцо.

Иерархический алгоритм В реальных кластерах топология обычно иерархическая. GPU внутри одной ноды имеют большую скорость передачи данных, чем GPU в разных нодах. Поэтому стоит учитывать это при выборе алгоритмов коммуникаций.

Например, внутри одной ноды мы можем построить два кольца по четыре девайса. А коммуникации между нодами можем выстроить как дерево, что позволит значительно снизить объем междоузовой передачи данных.